

前端开发技术报告

校园二手物品交易平台——转转

答辩人：阮崇轩

角色：前端开发工程师

技术栈：Vue3 + TypeScript + Vite + Element Plus + Pinia +
Axios

软件工程实训·课程设计答辩

华中农业大学·2026 年 7 月

目录

1 个人角色与工作概述	2
2 技术栈与工程架构	2
2.1 前端技术选型	2
2.2 项目目录结构	3
2.3 CSS 设计令牌体系	3
3 用户认证模块	4
3.1 登录页面	4
3.2 注册页面	4
3.3 找回密码页面	5
3.4 Axios 封装与 Token 自动刷新	5
4 消息通知系统	6
4.1 消息页面重写	6
4.2 通知铃铛组件	6
4.3 商品详情页联系卖家	7
5 订单详情页面	7
5.1 买家卖家双视角	7
5.2 发货物流录入	7
5.3 评价系统	7
6 UI 动效与交互优化	7
6.1 骨架屏加载	7
6.2 页面过渡动画	8
6.3 回到顶部按钮	8
6.4 导航栏滚动感知	8
6.5 按钮微交互	8
7 后端 Bug 修复贡献	8
7.1 验证码类型隔离（安全漏洞）	8
7.2 密码重置验证码提前消费（逻辑 Bug）	9
7.3 密码重置暴力破解防护	9
7.4 浏览量统计错误	9
7.5 订单通知联动	9

8 代码贡献统计	9
9 核心技术亮点	10

个人角色与工作概述

在本次软件工程实训项目中，我担任前端开发工程师，主要负责以下模块的开发工作：

- **前端工程搭建**：使用 Vue3 Composition API + TypeScript + Vite5 从零搭建项目，配置路由、状态管理、HTTP 客户端、UI 组件库等基础设施
- **用户认证模块**：登录页面、注册页面、找回密码页面的完整前端实现，包含 Token 管理、验证码机制、表单校验
- **消息通知系统**：站内信双栏聊天界面、系统通知铃铛组件、实时轮询机制、商品详情页联系卖家功能
- **订单详情页面**：买家卖家双视角操作、发货收货全流程、评价系统
- **UI 动效与交互优化**：骨架屏加载、页面过渡动画、滚动感知导航栏、按钮微交互、全局 CSS 设计令牌体系
- **后端 Bug 修复**：验证码类型隔离、防暴力破解、密码重置 Token 消费时序问题、浏览量统计错误

技术栈与工程架构

2.1 前端技术选型

本项目前端采用 Vue 3 Composition API 配合 TypeScript 开发，构建工具使用 Vite 5。选择这一技术栈的核心考量如下：

Vue 3 Composition API 相较于 Options API，提供了更好的逻辑复用能力和 TypeScript 支持。通过 `setup()` 语法糖，可以将相关的响应式状态、计算属性、方法组织在一起，而非分散在 `data`、`methods`、`computed` 等选项中。这在开发消息系统等复杂交互组件时尤为重要。

TypeScript 提供了静态类型检查，能够在编译阶段捕获大量潜在错误。虽然初始开发成本略高，但对于一个 5 人团队协作的项目，类型系统显著降低了前后端接口联调时的沟通成本——API 返回数据的结构通过类型定义形成契约。

Vite 5 基于浏览器原生 ES Module 的开发服务器实现了极快的热更新，大型项目的冷启动时间通常在 1 秒以内。其生产构建使用 Rollup，配合 Tree Shaking 和代码分割，优化了最终包体积。

2.2 项目目录结构

zhuanzhuan-frontend/

```
src/
  api/          # API 接口封装（按模块划分）
  assets/       # 静态资源
  components/   # 通用组件
  layouts/      # 布局组件（MainLayout, AdminLayout）
  router/       # Vue Router 路由配置
  stores/       # Pinia 状态管理
  types/        # TypeScript 类型定义
  utils/        # 工具函数（auth, request, productImage）
  views/        # 页面组件（按功能模块分组）
```

2.3 CSS 设计令牌体系

我设计了一套基于 CSS Variables 的全局主题系统，定义了 15 个设计令牌变量，实现了品牌色、间距、圆角、阴影的全局统一管理：

```
:root {
  --c-primary: #2D6A4F;          /* 主色-森林绿 */
  --c-primary-dark: #1B4332;     /* 深色变体 */
  --c-primary-light: #52B788;    /* 浅色变体 */
  --c-primary-bg: #E8F5E9;       /* 浅绿背景 */
  --c-accent: #E76F51;           /* 强调色-暖橙 */
  --c-accent-hover: #D65A3E;     /* 强调色悬停 */
  --c-warm: #F4A261;             /* 暖色点缀 */
  --c-bg: #FDFBF7;              /* 全局背景 */
  --c-surface: #FFFFFF;          /* 卡片/面板背景 */
  --c-text: #1F2937;            /* 正文颜色 */
  --c-text-secondary: #6B7280;   /* 次要文字 */
  --c-muted: #9CA3AF;           /* 弱化文字 */
  --c-border: #E5E7EB;          /* 边框 */
  --radius-sm: 8px; --radius-md: 12px; --radius-lg: 16px;
  --shadow-sm/md/lg: ...;       /* 三级阴影 */
}
```

这套令牌体系的核心价值在于：通过覆盖 Element Plus 的 CSS 变量，实现了对官方组件库的全局样式统一。例如，所有按钮 `el-button--primary` 的背景色被覆盖为 `var(--c-primary)`，确保了品牌一致性。21 个页面、数百个组件实例共享同一套

设计语言，修改一个变量即可全局生效。

用户认证模块

3.1 登录页面

登录页面实现了渐变绿色背景配合浮动几何圆形装饰的视觉设计。技术要点包括：

表单校验：使用 Element Plus 的 `el-form` 配合自定义 `rules` 实现前端校验。账号字段支持用户名、邮箱、手机号三种登录方式，密码字段支持显示/隐藏切换。

Token 管理：登录成功后，后端返回 Access Token (2 小时有效) 和 Refresh Token (7 天有效)。前端通过 Pinia store 管理登录状态，将 Token 存储到 `localStorage` 中：

```
async function login(credentials: any) {
  const res = await userApi.login(credentials)
  token.value = res.data.accessToken
  setToken(res.data.accessToken)
  setRefreshToken(res.data.refreshToken)
  userInfo.value = res.data.user
  setUserInfo(res.data.user)
  return res.data
}
```

3.2 注册页面

注册页面实现了邮箱验证码注册流程。关键实现细节：

验证码倒计时：点击“获取验证码”按钮后进入 60 秒冷却期，按钮文字显示剩余秒数，防止用户重复点击导致后端频率限制或邮件轰炸。

```
let count = 60
codeBtnText.value = `${count}s`
const timer = setInterval(() => {
  count--
  codeBtnText.value = `${count}s`
  if (count <= 0) { clearInterval(timer); ... }
}, 1000)
```

前端校验：用户名正则校验 (2-20 位字母数字下划线)、密码强度校验 (8-20 位含字母和数字)、验证码非空校验。

3.3 找回密码页面

这是一个原创实现的两步流程页面：

Step 1: 输入注册邮箱 → 点击获取验证码 → 输入收到的验证码 → 点击”下一步”

Step 2: 输入新密码 → 确认新密码 → 密码强度校验 → 两次密码一致性校验 → 提交

核心设计思想：将验证步骤和密码设置步骤分离，避免单页信息过载。每一步只关注一个核心任务，降低用户认知负担。

3.4 Axios 封装与 Token 自动刷新

这是前端工程中最关键的基础设施之一。我封装了一个 Axios 实例，实现了以下功能：

请求拦截器：自动从 localStorage 读取 Token 并注入到请求头的 Authorization 字段（Bearer Token 格式）。

响应拦截器——Token 自动刷新：当收到 401 状态码时，自动使用 Refresh Token 换取新的 Access Token。这是一个教科书级别的并发控制实现：

```
let isRefreshing = false
let failedQueue: Array<{resolve, reject}> = []

// 当多个请求同时遇到401时，只发起一次刷新请求
if (isRefreshing) {
  return new Promise((resolve, reject) => {
    failedQueue.push({ resolve, reject })
  }).then(token => {
    originalRequest.headers.Authorization = `Bearer ${token}`
    return request(originalRequest)
  })
}
isRefreshing = true
// 发起刷新请求...
```

核心原理：使用 isRefreshing 标志位确保多个并发请求只触发一次 Token 刷新。其余请求被放入 failedQueue 队列等待，刷新成功后批量重试。这避免了并发 401 导致的多次刷新和潜在的 Token 竞争问题。

403 过期登录处理：当收到 403 且本地存在 Token 时，说明 Token 指向的用户已被删除或禁用，自动清除登录状态并跳转到登录页。

错误分类提示：区分网络超时（ECONNABORTED）、网络断开（无 response）、

服务器错误（5xx）、权限不足（403）、资源不存在（404）、请求过频（429）等多种错误类型，给用户提供明确的错误信息。

消息通知系统

4.1 消息页面重写

原始的消息页面仅有 86 行基础布局代码。我将其完全重写为一个 300+ 行的现代化聊天界面，包含以下核心特性：

双栏布局：左侧 340px 会话列表 + 右侧自适应聊天面板，使用 CSS Grid 实现：

```
.msg-layout {  
  display: grid;  
  grid-template-columns: 340px 1fr;  
  height: calc(100vh - 140px);  
  border-radius: 16px;  
  overflow: hidden;  
}
```

聊天气泡设计：我方消息使用主色背景、白色文字、右对齐、右下角小圆角；对方消息使用灰色背景、深色文字、左对齐、左下角小圆角。这种视觉区分让用户一眼就能分辨消息来源。

实时轮询：使用 JavaScript `setInterval` 实现每 5 秒自动拉取当前会话的新消息。关键技术点在于使用 `onUnmounted` 生命周期钩子清理定时器，防止内存泄漏。

URL 参数自动开聊：通过 Vue Router 的 `route.query.to` 参数，支持从商品详情页一键跳转到与卖家的聊天窗口。在页面 `onMounted` 时检测 URL 参数，自动打开对应会话甚至创建新会话。

搜索会话：使用 `computed` 属性实现会话列表的实时过滤搜索。

日期分隔线：当两条消息不在同一天时，在中间插入日期标签，提升长时间对话的可读性。

4.2 通知铃铛组件

在页面头部（MainLayout）中新增了一个通知铃铛图标。核心实现：

Popover 下拉面板：使用 Element Plus 的 `el-popover` 组件，点击铃铛图标展开通知列表。列表最多显示 10 条通知，每条通知包含标题、内容、时间。

未读红点：使用 `el-badge` 组件显示未读通知数量，每 30 秒自动轮询后端 `/notification/unread` 接口更新数字。

点击跳转：点击通知项根据其类型（订单/消息/系统）跳转到对应的页面，实现了通知到操作的闭环。

4.3 商品详情页联系卖家

在商品详情页的卖家信息卡片中添加了一个绿色的”联系卖家”按钮。点击后跳转到 `/message?to=sellerId`，消息页面会自动打开与该卖家的聊天窗口。未登录用户点击后会先跳转到登录页。

订单详情页面

5.1 买家卖家双视角

订单详情页根据当前登录用户自动判断角色(通过比较当前用户 ID 与订单的 `buyerId/sellerId`)，展示不同的操作按钮：

- **买家视角**：待付款时显示”去支付”和”取消订单”按钮；待收货时显示”确认收货”按钮；已完成时显示”去评价”按钮
- **卖家视角**：待发货时显示”录入物流发货”按钮

需要特别指出的是，后端 `OrderVO` 原本没有 `buyerId` 和 `sellerId` 字段，无法支持前端角色判断。我发现了这个问题后，在后端 `OrderVO` 中添加了这两个字段，并在 `OrderServiceImpl` 的 `batchToOrderVO` 方法中进行了赋值。

5.2 发货物流录入

卖家发货时，弹出一个对话框，以彩色卡片网格的形式展示 8 家物流公司（顺丰、中通、圆通、韵达、申通、EMS、京东、极兔），每家使用品牌识别色。用户选择后输入快递单号即可完成发货。

5.3 评价系统

交易完成后，买家可以对订单进行评价：使用 `El-Rate` 组件进行 1-5 星评分，配合文本输入框填写评价内容。提交后对接后端 `/order/id/review` 接口。

UI 动效与交互优化

6.1 骨架屏加载

首页商品列表加载时，先展示 8 个灰色占位卡片，使用 `CSS Shimmer` 动画（渐变色从左到右移动的光效），给用户即时的视觉反馈，降低等待焦虑。数据加载完成后，骨架屏消失，真实卡片以 `fadeInUp` 动画交错入场。

实现原理：使用 CSS 动画配合 `background: linear-gradient(90deg, ...)` 和 `background-size: 200%` 实现光效移动。

6.2 页面过渡动画

在 `App.vue` 中使用 Vue Router 的 `router-view` 插槽配合 `<transition>` 组件，实现了路由切换时的 fade（淡入淡出）过渡效果。设置 `mode="out-in"` 确保旧页面完全离开后再进入新页面。

6.3 回到顶部按钮

使用 Vue 的响应式系统监听 `window.scrollY`，当滚动超过 400px 时，在右下角显示一个绿色的圆形回到顶部按钮，点击后使用 `window.scrollTo(top:0, behavior:'smooth')` 平滑滚动到顶部。按钮的出现和消失使用 CSS transition 实现弹性缩放动画。

6.4 导航栏滚动感知

在 `MainLayout` 中，通过监听 `window.scrollY`，当页面滚动超过 10px 时，为导航栏添加阴影和毛玻璃模糊效果（`backdrop-filter: blur(12px)`），提供视觉层次感。

6.5 按钮微交互

全局覆盖了 Element Plus 按钮的默认行为：hover 时按钮上浮 1px 并增加阴影，active 时按钮下压回弹，所有变化使用 cubic-bezier 缓动函数实现平滑过渡。

后端 Bug 修复贡献

虽然我的主要角色是前端开发，但我也发现并修复了后端的多个安全和逻辑问题：

7.1 验证码类型隔离（安全漏洞）

原始代码中，`sendCode` 方法使用 `captcha:{target}` 作为 Redis Key 存储验证码，未区分验证码的用途（注册/重置密码）。这意味着一个注册验证码可以被用于重置密码，存在安全漏洞。

修复方案：将 Redis Key 改为 `captcha:{type}:{target}` 格式，注册码和重置码存储在独立的命名空间中，互不可用。同时更新了 `register` 和 `resetPassword` 方法中的 Key 匹配逻辑。

7.2 密码重置验证码提前消费（逻辑 Bug）

原始代码在验证码校验通过后立即执行 `redisTemplate.delete(cacheKey)`，然后才进行“新旧密码是否相同”等后续校验。如果后续校验失败，验证码已被删除，用户需要重新获取验证码，体验极差。

修复方案：将 `delete(cacheKey)` 移至整个方法的最末尾，确保只有在密码真正重置成功后才删除验证码。注册流程中的同类问题也一并修复。

7.3 密码重置暴力破解防护

为 `resetPassword` 方法新增了频率限制：同一邮箱 5 次/15 分钟，使用 Redis Key `pwd:reset:rate:{target}` 计数。同时新增了邮箱格式校验和空值检查。

7.4 浏览量统计错误

原始代码中，卖家查看自己发布的商品也会增加浏览量。修改为仅在非卖家浏览时才执行 `incrementViewCount` 操作。

7.5 订单通知联动

在 `MessageServiceImpl.sendMessage` 中增加了自动创建系统通知的逻辑。每次发消息时，自动为接收方创建一条通知。在 `OrderServiceImpl` 的各状态流转方法（支付、发货、收货）中也增加了通知触发逻辑。

代码贡献统计

模块	代码行数（估计）
用户认证（Login/Register/ForgotPassword）	约 800 行
消息通知系统（MessagePage/MainLayout）	约 700 行
订单详情（OrderDetail）	约 600 行
UI 动效系统（App.vue/HomePage/全局 CSS）	约 500 行
Axios 封装（request.ts）	约 120 行
后端 Bug 修复（Java）	约 100 行
合计	约 2,820 行

核心技术亮点

1. **Token 队列刷新机制**：多请求并发 401 时只发一次刷新请求，其余排队等待。这是一个经典的并发控制模式，确保了 Token 刷新过程的原子性。
2. **验证码三段式安全防护**：类型隔离（不同用途不可互用）+ 频率限制（防暴力破解）+ 防消费（校验失败不删除），构成了完整的验证码安全体系。
3. **CSS 设计令牌体系**：15 个 CSS Variables 实现 21 个页面、数百个组件的风格统一，通过覆盖 Element Plus 原生变量实现第三方组件库的深度定制。
4. **骨架屏 + 交错动画**：骨架屏 Shimmer 动画提供即时加载反馈，数据就绪后以瀑布流交错动画入场，显著提升了感知性能。
5. **消息通知实时化**：5秒聊天轮询 + 30 秒通知轮询 + 订单状态自动通知联动，构建了完整的前端实时通信方案。

消息系统实时通信原理

消息系统的实时性是通过前端轮询（Polling）机制实现的。选择轮询而非 WebSocket 的考量：项目规模决定了消息量较低，轮询的简单性和可靠性更适合当前场景，且无需额外的服务端 WebSocket 配置。

轮询实现：使用 JavaScript 的 `setInterval` 函数，每 5 秒向 `/message/conversation/{userId}` 端点发起 GET 请求，将返回的消息列表与当前列表进行长度比较，如果有新消息则更新 UI 并滚动到底部。

性能优化：轮询仅在用户打开了某个会话时进行（`selectedUserId !== null`），避免无效请求。轮询间隔 5 秒是在实时性和服务器负载之间的平衡选择。消息通知的轮询间隔为 30 秒，因为通知的时效性要求低于聊天消息。

定时器生命周期管理：在 `onMounted` 中启动定时器，在 `onUnmounted` 中清理。这防止了用户离开页面后定时器继续运行导致的内存泄漏和无效网络请求。这是 Vue 3 Composition API 中生命周期管理的一个典型应用。

订单状态机设计

订单详情页的核心是一个有限状态机。订单有 5 种状态：待付款、待发货、待收货、已完成、已取消。状态之间的转换遵循严格的规则：

- 待付款 → 待发货（买家支付后） / 已取消（买家取消）
- 待发货 → 待收货（卖家发货后）

- 待收货 → 已完成（买家确认收货后）
- 已取消和已完成是终态，不可再转换

前端通过 `computed` 属性动态计算当前用户角色（通过比较 `currentUser.userId` 与 `order.buyerId/sellerId`），然后根据角色和订单状态组合决定显示哪些操作按钮。这是一个典型的“状态 × 角色”二维决策矩阵。

订单进度条实现：定义了一个 `steps` 数组（待付款、待发货、待收货、已完成），通过 `currentStep` 计算属性根据订单状态确定当前步骤索引。通过 CSS 类名 `done/active/pending` 控制各步骤的视觉样式——已完成步骤显示绿色填充圆 + 勾选图标、当前步骤显示绿色脉冲动画圆 + 数字、未完成步骤显示灰色空心圆 + 数字。

支付系统设计

支付页面实现了四步流程的模拟支付体验。这不是真正的支付集成，而是对支付流程的前端模拟，用于演示完整的交易闭环。

Step 1——选择支付方式：展示订单摘要（商品名、订单号、金额）和三种支付方式卡片（微信支付绿色渐变、支付宝蓝色渐变、校园卡绿色渐变）。用户选择后点击“确认支付”。

Step 2——处理中动画：显示旋转的 CSS `spinner` 配合点状动画（“正在连接支付网关…”）。使用 `setTimeout` 模拟 1.5-2.5 秒的支付处理延迟，然后调用后端 `PUT /order/{id}/pay` 接口。

Step 3——成功状态：使用 SVG 绘制勾选图标，配合 `stroke-dashoffset` 动画实现勾选线条的绘制效果。显示支付金额确认，2 秒倒计时后自动跳转到订单详情页。

Step 4——失败状态：显示抖动动画的 X 图标、“支付失败”文字和错误信息，提供“重试”和“返回订单”两个按钮。使用 CSS `@keyframes shake` 实现左右抖动效果。

密码重置安全设计

找回密码功能虽然看起来简单，但涉及多个安全设计要点：

两步验证流程：将邮箱验证和密码重置分为两个步骤，确保用户在验证身份后才能设置新密码。这种分步设计不仅提升了安全性，也改善了用户体验（每步只关注一件事情）。

验证码安全：验证码发送时带有类型标签（`reset`），存储在 Redis 中与注册验证码（`register`）隔离。验证码 5 分钟有效，发送频率限制为每 5 分钟 3 次。

密码强度要求：新密码必须满足 8-20 位且包含字母和数字的正则校验。前后端双重校验确保数据一致性。

防消费机制：验证码只在密码重置成功后才删除。如果中间任何校验失败（如新旧密码相同），验证码不会被消耗。这意味着用户不需要因为密码不符合要求而重新获取验证码。

前端安全实践

除了后端的 JWT、BCrypt、频率限制等安全措施，前端也实现了多层安全防护：

XSS 防护：商品描述可能包含用户输入的 HTML。通过 `sanitizedDescription` 计算属性实现三重过滤——正则移除 `<script>` 和 `<style>` 标签、剥离所有 HTML 标签、使用 DOM `textarea` 的 `innerHTML` $\rightarrow$ `value` 转换解码 HTML 实体。

Token 安全：Access Token 存储在 `localStorage` 中并在每次请求时自动注入。Token 过期后的刷新过程使用队列机制防止并发冲突。登出时调用后端接口将 Token 加入 Redis 黑名单。

路由守卫：在 `router.beforeEach` 中实现全局路由守卫。检查 `meta.requiresAuth` 标志，未登录用户重定向到登录页。检查 `meta.requiresAdmin` 标志，非管理员重定向到首页。

输入校验：所有表单都实现了前端校验规则（Element Plus Form Rules），包括必填检查、格式校验（邮箱、手机号、密码强度）、长度限制。前端校验不能替代后端校验，但能提供即时反馈和减少无效请求。

项目中的技术决策与权衡

为什么使用 Pinia 而非 Vuex：Pinia 是 Vue 3 官方推荐的状态管理库，提供了更好的 TypeScript 支持和更简洁的 API。本项目只有一个 store（user），管理登录状态、用户信息、Token。对于这种中等复杂度的应用，Pinia 的模块化设计和 devtools 集成提供了足够的开发体验。

为什么使用 Element Plus 而非从头写 CSS：Element Plus 提供了 80+ 组件，覆盖了本项目需要的所有 UI 元素（表单、表格、对话框、标签页、分页等）。使用成熟的组件库可以大幅减少开发时间，但需要通过 CSS 变量覆盖来定制主题以匹配品牌设计。

为什么使用 CSS Variables 而非 CSS-in-JS：CSS Variables 是浏览器原生支持的动态样式方案，无需额外的运行时开销。通过 `:root` 中定义的变量，可以在全局范围内统一修改主题色、间距等设计参数。Element Plus 也从 2.x 版本开始支持 CSS 变量覆盖。

消息轮询 vs WebSocket 的决策：对于校园二手交易平台这种非实时性要求极高的应用场景，轮询已足够满足需求。WebSocket 需要额外的服务端配置（Spring WebSocket + STOMP），增加了部署复杂度。5 秒的轮询间隔在用户体验和服务器负载之

间取得了平衡。

遇到的问题与解决方案

问题一：Docker时区导致时间显示错误。Docker 容器默认使用 UTC 时区，而中国使用 UTC+8。导致前端显示的所有时间都晚 8 小时。解决：在 docker-compose.yml 中为 MySQL 和 Backend 容器添加 TZ: Asia/Shanghai 环境变量。

问题二：旧 Token 导致全站 403。重新导入种子数据后，浏览器中存储的旧用户 Token 对应的用户已被删除，JWT 过滤器拒绝该 Token，所有需要认证的页面返回 403。解决：在前端 Axios 响应拦截器中，检测到 403 且本地存在 Token 时，自动清除登录状态并跳转登录页。

问题三：CSS 设计令牌的跨组件一致性。21 个页面分布在不同文件中，如果没有统一的设计令牌体系，每个开发者会自行选择颜色和间距，导致风格不一致。解决：在 App.vue 的 :root 中定义全局 CSS Variables，所有组件通过 var(--c-primary) 等变量引用，确保全局一致性。

问题四：Element Plus 组件样式覆盖。Element Plus 的组件使用了自己的 CSS 变量和类名体系。直接修改组件内部样式需要使用 :deep() 或 !important，容易造成样式冲突。解决：通过覆盖 Element Plus 的 CSS 变量（如 --el-button-bg-color）实现全局主题定制，避免直接修改组件内部样式。